

DAILY TASK PLAN — LAYER BY LAYER

Sathish & Karthik · Build order follows dependency chain, not the diagram left-to-right

Why NOT left-to-right (diagram order)?	Why THIS order instead?
Tool Layer (col 1) needs the Backend (col 2) to exist first — adapters write findings INTO FastAPI + PostgreSQL. You cannot build an adapter with nowhere to store results.	Data Store FIRST — because everything writes to it. Backend SECOND — the central glue that uses the DB. Tools THIRD — adapters now have a backend to feed into. External APIs FOURTH — enrich findings now in the DB. Frontend LAST — calls fully working API endpoints.

Build Order	Layer	Technologies	Depends On	Approx Days
1st — Build First	DATA STORE	PostgreSQL, Redis, MinIO, Alembic migrations	Nothing — foundation layer	Days 1–3
2nd	BACKEND skeleton	FastAPI, Celery, JWT auth, Pydantic models, SQLAlchemy	Data Store must exist	Days 4–10
3rd	TOOL LAYER	Nuclei, Nmap, Trivy, Gitleaks, Prowler, Checkov adapters	Backend + DB schema ready	Days 11–22
4th	EXTERNAL APIS	NVD, EPSS, CISA KEV, MITRE STIX, Claude API	Tool findings exist in DB to enrich	Days 23–34
5th — Build Last	FRONTEND / OUTPUT	React, Recharts, Tailwind, PDF, Slack, Jira webhooks	All API endpoints fully working	Days 35–50

Person	Role	Primary Layer
Sathish ■	Security + Backend	Data Store, Tool Adapters, External APIs (AI + Intel)
Karthik ■	Full Stack	Backend skeleton (FastAPI routes), Frontend (React), Docker + deployment
Both ■	Together	Initial setup, integration testing, DB schema review, production deploy

PHASE 1

DATA STORE — Build First

PostgreSQL · Redis · MinIO · Alembic · Docker Compose

Everything else writes to or reads from this layer. Getting the schema right here avoids painful migrations later. Both of you work together on Days 1–3 — this sets up the shared foundation.

Day	Sathish ■	Karthik ■	■ Done When
Day 1 Tomorrow	Install: Python 3.12, Docker Desktop, VS Code, git. Run: docker run hello-world (confirm Docker works). Create project GitHub repo with folders: backend/ frontend/	Install: Node.js 20 LTS, VS Code, git. Confirm: node -v and npm -v work. Clone the GitHub repo Sathish just created.	Both can run hello-world. Shared GitHub repo exists with correct structure.
Day 2	Write docker-compose.yml with ONLY these 3 services: postgres:16-alpine (port 5432) redis:7-alpine (port 6379) minio/minio (ports 9000, 9001) Run: docker-compose up -d → confirm all 3 start.	Write .env.example file with all required keys: DATABASE_URL, REDIS_URL, ANTHROPIC_API_KEY, JWT_SECRET, MINIO_USER, MINIO_PASSWORD Add .env to .gitignore. Copy .env.example → .env locally.	docker-compose ps shows 3 services Running. .env.example committed. .env never committed.
Day 3	pip install sqlalchemy alembic psycopg2-binary pydantic Write 5 SQLAlchemy models (Python classes = DB tables): orgs, users, assets, scans, findings Run alembic init + write migration. Run: alembic upgrade head. Verify: docker exec postgres psql -U user platform ldt	Write backend/app/database.py: engine = create_engine(DATABASE_URL) SessionLocal = sessionmaker(...) get_db() dependency for FastAPI Test: Python script that inserts a test org row and reads it back.	All 5 tables visible in PostgreSQL. Python can insert + read a row from the DB.

Phase 1 complete signal: docker-compose ps shows all 3 services green. alembic upgrade head runs without errors. A Python script can INSERT and SELECT from the findings table.



BACKEND SKELETON — Central Glue

FastAPI · Celery · JWT Auth · Pydantic · SQLAlchemy

This phase builds the API that every other layer communicates through. No tool adapters yet — just the skeleton: endpoints, auth, job queue, and a mock scan task that confirms the full pipeline works end-to-end.

Day	Sathish ■	Karthik ■	■ Done When
Day 4	<code>pip install fastapi uvicorn</code> Write <code>backend/app/main.py</code>: <code>app = FastAPI(title='Security Platform')</code> <code>GET /health → {status: ok, version: 0.1}</code> Run: <code>uvicorn app.main:app --reload → test in browser at localhost:8000/docs</code>	Add FastAPI to <code>docker-compose.yml</code>: build: <code>./backend</code>, ports: <code>8000:8000</code> volumes: <code>./backend:/app</code> (hot reload) Test: <code>curl localhost:8000/health</code> returns JSON.	FastAPI <code>/health</code> works. Swagger docs at <code>localhost:8000/docs</code>.
Day 5	<code>pip install python-jose[cryptography]</code> <code>passlib[bcrypt]</code> Write auth module: <code>POST /auth/token (email+password → JWT)</code> <code>GET /auth/me (token → {id, email, role})</code> <code>create_access_token()</code>, <code>verify_token()</code> helpers	Write Pydantic request/response schemas: <code>LoginRequest</code>, <code>TokenResponse</code>, <code>UserOut</code> <code>ScanRequest</code>, <code>ScanResponse</code>, <code>FindingOut</code> These are the data contracts between frontend and backend.	<code>POST /auth/token</code> with valid user → returns JWT. <code>GET /auth/me</code> with JWT → returns user info.
Day 6	Add user + org management endpoints: <code>POST /orgs</code> (create org) → admin only <code>POST /users</code> (create user in org) RBAC decorator: <code>viewer / analyst / admin</code> roles Add <code>org_id</code> filter to ALL DB queries (multi-tenant isolation).	Test RBAC thoroughly: <code>viewer</code> trying <code>analyst</code> endpoint → 403 Forbidden <code>analyst</code> trying <code>admin</code> endpoint → 403 Forbidden Write Postman / Bruno collection to test all auth flows.	2 test orgs created. User from Org A cannot access Org B data. All 3 roles tested and working correctly.
Day 7	<code>pip install celery[redis]</code> Write <code>celery_app.py</code>: connect to Redis broker Write first real task: <code>@celery.task</code> <code>def run_scan_task(target, scan_types, org_id):</code> <code>time.sleep(5)</code> # mock for now <code>return {status: complete}</code>	Add Celery worker to <code>docker-compose.yml</code>: command: <code>celery -A app.celery worker</code> <code>--loglevel=info</code> <code>depends_on: [redis, db]</code> Test: call task from Python shell, check it runs in worker logs.	Celery worker starts. Mock task runs. Worker logs show task received + completed.
Day 8	Write scan endpoints: <code>POST /scans</code> → pushes Celery task → returns <code>{job_id}</code> <code>GET /scans/{id}</code> → reads job status from Redis <code>GET /scans/{id}/findings</code> → reads findings from PostgreSQL Store job state in Redis hash: <code>{status, progress, org_id}</code>	Write asset endpoints: <code>POST /assets</code> (register URL, IP, or repo) <code>GET /assets</code> (list org assets) <code>PATCH /assets/{id}</code> (update <code>asset_weight</code>: 0.5/1.0/1.5/2.0) Test all endpoints with Postman.	<code>POST /scans</code> returns <code>job_id</code>. <code>GET /scans/{id}</code> returns status: <code>queued</code> → <code>running</code> → <code>complete</code>.
Day 9	Add SSE (Server-Sent Events) progress stream: <code>GET /scans/{id}/progress</code> → <code>EventSource</code> stream Worker updates progress% in Redis every 10 seconds SSE reads Redis and streams to client	Add webhook registration: <code>POST /webhooks</code> (register callback URL) Celery task: <code>POST</code> to webhook on scan completion Retry 3x with exponential backoff on failure.	SSE progress stream updates correctly (test with <code>curl --no-buffer</code>). Webhook receives <code>POST</code> when mock scan completes.
Day 10	Backend review day: Run all Postman tests. Check every endpoint returns correct status codes. Add missing input validation (Pydantic rejects invalid targets). Add rate limiting: <code>pip install slowapi</code> → 10 scans/hr per user.	Docker Compose review: <code>docker-compose down</code> → <code>docker-compose up -d</code> → confirm everything restarts cleanly. Write <code>README.md</code>: how to run the project locally in 5 commands.	All 8 API endpoints work. Auth is enforced everywhere. Rate limiting returns 429 after 10 scans/hour.



TOOL LAYER — The Scanners

Nuclei · Nmap · Trivy · Gitleaks · Prowler · Checkov

Now that the backend exists, wire in the real scanners one by one. Each adapter follows the same pattern: download binary → write Python wrapper → subprocess.run() → parse JSON output → return list of Finding objects. Karthik adds each new scan type to the UI as Sathish completes each adapter.

Day	Sathish ■ — Tool Adapter	Karthik ■ — UI / Integration	■ Done When
Day 11	Download Nuclei binary (github.com/projectdiscovery/nuclei/releases). Place in backend/tools/nuclei. Make executable: <code>chmod +x nuclei</code> Run manually: <code>./nuclei -target https://example.com -t cves -json</code> Read and understand the JSON output structure.	Build Scan Launch page in React: URL/IP input field Scan type checkboxes: Vuln / Secret / SCA / Cloud POST to /scans on submit → show job_id	Nuclei runs and produces JSON output manually. Scan launch form POSTs to backend and gets job_id back.
Day 12	Write NucleiAdapter class (backend/app/adapters/nuclei.py): <code>def run(target, templates) → list[Finding]</code> <code>subprocess.run(['nuclei', '-target', target, '-json', ...])</code> parse each stdout line as JSON → Finding Pydantic object	Build Findings Table in React: Poll GET /scans/{id}/findings every 5s Columns: CVE ID, Severity badge (coloured), Asset, URL Default sort: severity descending	Nuclei adapter returns real Finding objects. Findings table shows results from a real scan.
Day 13	Download Nmap. Install: <code>apt install nmap</code> (or <code>brew install nmap</code>). Write NmapAdapter class: <code>subprocess.run(['nmap', '-sV', '-oX', '-', target])</code> Parse XML → {port, service, version} per open port Store as asset fingerprint (used later for FP filtering)	Add progress bar using SSE: EventSource on GET /scans/{id}/progress Show 0%→100% live bar instead of polling Show spinner while queued, bar while running.	Nmap adapter returns open ports + service versions. Progress bar updates live during a running scan.
Day 14	Write TrivyAdapter (container/SCA scan): <code>subprocess.run(['trivy', 'image', '--format', 'json', image_name])</code> Parse: CVE ID, package name, installed version, fixed version Test: trivy image nginx:1.24 → should find multiple CVEs	Add Docker image input to scan launch: Show only when SCA scan type is checked Display package name + fixed version in finding detail Add 'Upgrade to: X.X.X' badge	Trivy scans a real Docker image. Package CVEs with fix versions appear in UI.
Day 15	Write GitleaksAdapter (secret scanning): Clone repo to temp folder: <code>git clone {repo_url} /tmp/scan_{id}</code> <code>subprocess.run(['gitleaks', 'detect', '--source', path, '-f', 'json'])</code> Parse: secret_type, file, line_number, commit, partial_secret	Add repo URL input to scan launch: Show when Secret Scan type is checked Add LIVE SECRET badge (red) for verified=True findings Show file path + commit hash in finding detail.	Gitleaks scans a real test repo. Secrets appear in UI with file path and commit.
Day 16	Add TruffleHog for secret verification: pip install trufflehog (or download binary) Run after Gitleaks on same repo Merge + deduplicate by secret fingerprint hash Add verified (bool) field — TruffleHog calls provider API	Deduplication visible in UI: Same secret from 40 commits = 1 finding, not 40 verified=True shows ■ LIVE badge verified=False shows ■■ UNVERIFIED badge	Both secret scanners run. Deduplication works. Live secrets correctly flagged.

PHASE 4

EXTERNAL APIs — Intelligence & AI

NVD · EPSS · CISA KEV · MITRE STIX · Claude API (Anthropic)

Tool findings now exist in the database. This phase enriches every finding with real-world threat intelligence and AI-generated reasoning. Sathish owns this entire phase — it is the core of your platform's value-add.

Day	Sathish ■ — Intelligence + AI	Karthik ■ — UI for AI features	■ Done When
Day 23	Write NVDClient (backend/app/intel/nvd.py): GET https://services.nvd.nist.gov/rest/json/cves/2.0?cveId={id} Extract: cvss_base_score, cvss_vector, description, cwe_id[] Cache in Redis: SET nvd:{cve_id} {json} EX 86400 (24h TTL)	Add CVSS score column to findings table. Show CVSS vector string on hover (tooltip). Add CWE category tag to finding detail.	NVD data cached in Redis. CVSS scores appear in findings table. Cache hit confirmed (second call returns in <1ms).
Day 24	Write KEVClient (backend/app/intel/kev.py): Download CISA KEV JSON: https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json Load all CVE IDs into Redis SET: SADD kev:cves {cve_id} Lookup: SISMEMBER kev:cves {cve_id} → bool (O(1)) Refresh daily via Celery Beat scheduled task.	Add KEV badge to findings table: ■ EXPLOITED IN WILD for kev_listed=True Show KEV due date in finding detail KEV findings always sort to top of table.	CISA KEV loaded into Redis. KEV badge shows for known CVEs. O(1) lookup confirmed in logs.
Day 25	Write EPSSClient (backend/app/intel/epss.py): GET https://api.first.org/data/v1/epss?cve={id} Batch up to 100 CVEs per request (efficiency) Cache: SET epss:{cve_id} {score} EX 86400 Add epss_score (float 0–1) to Finding	Add EPSS score to finding detail: Show as percentage: '94th percentile' Add colour bar: green (0–30%) → amber (30–70%) → red (70–100%) Tooltip: 'Probability this CVE is exploited in wild'	EPSS scores appear with colour bar. Batch fetch confirmed (100 CVEs = 1 API call).
Day 26	Compute priority score for every finding: score = epss × cvss × (2.0 if kev else 1.0) × asset_weight asset_weight set per asset: 0.5/1.0/1.5/2.0 Store priority_score + priority_rank (1=highest) in DB Top 10 by score = immediate action list	Make priority_score the default sort in findings table. Add priority rank badge: #1 #2 #3 ... next to top 10 findings. Add 'Top 10 — Fix Immediately' section at top of scan results.	Findings sorted by priority_score. Top 10 highlighted. Rank #1 finding has highest real-world exploitation risk.
Day 27	Download MITRE ATT&CK; STIX bundle: curl -o backend/data/attack.json https://raw.githubusercontent.com/mitre/cti/master/enterprise-attack/enterprise-attack.json pip install stix2 Load at Celery worker startup: MemoryStore(bundle)	Add ATT&CK; technique badge to finding detail: Show: T1190 — Exploit Public-Facing Application Show tactic name: Initial Access / Execution / etc. Link to MITRE ATT&CK; website page.	STIX loads at worker startup (check logs: 'ATT&CK; bundle loaded'). Technique ID and tactic appear in finding detail.
Day 28	Write MITREMapper.map_cve(cve_id): Query MemoryStore for CVE → technique relationships Fallback: if no direct match, use CWE → technique mapping table Add technique_id, technique_name, tactic to Finding	Add kill-chain position indicator to finding detail: Show mini kill-chain: Recon → Init Access → Exec → ... (current stage highlighted) Grey out stages before and after current finding's tactic.	MITRE mapping works for common CVEs. Kill-chain position shows correctly in finding detail.



FRONTEND / OUTPUT — Polish & Deploy

React · Recharts · Tailwind · Docker · Compliance · Production

The backend is fully working. Phase 5 polishes the UI, adds compliance reports, implements recurring scans, and deploys to production for the first client. Karthik leads this phase — Sathish supports with backend fixes as needed.

Day	Sathish ■ — Backend support	Karthik ■ — Frontend + Deploy	■ Done When
Day 35	Add OWASP Top 10 tagging: Nuclei templates tagged A01–A10 in their metadata Parse template tags → add <code>owasp_category</code> to Finding Build <code>owasp_coverage</code> dict per scan (count per category)	Build OWASP coverage heatmap: 10 boxes (A01–A10) on dashboard Green=0 findings, Amber=1–2, Red=3+ Click box → filter findings table to that OWASP category	OWASP heatmap shows correct counts. Clicking a box filters the table.
Day 36	Add compliance framework mapping: Prowler findings already tagged with CIS/PCI-DSS/SOC2/HIPAA control IDs Build <code>compliance_report(scan_id, framework)</code> → control pass/fail list POST <code>/scans/{id}/compliance-report?framework=pci</code>	Build compliance report page: Framework selector (PCI-DSS / HIPAA / SOC2 / CIS / GDPR) Table: control ID, description, PASS/FAIL, finding count Export PDF button for compliance report	Compliance table shows correct pass/fail. PDF export works.
Day 37	Add scan scheduling (recurring): Add schedule field to Asset: daily/weekly/monthly/manual pip install celery[redis] — enable Celery Beat Beat task: check assets with due schedules → trigger scan	Add schedule selector to asset settings: Dropdown: Manual / Daily / Weekly / Monthly Show: Next scheduled scan: Tomorrow 02:00 Show: Last scanned: 2 hours ago	Scheduled scan auto-triggers at correct time. Next scan time shows correctly in asset settings.
Day 38	Add scan history page backend: GET <code>/scans</code> (list all scans for org, paginated) Include: scan date, target, finding counts by severity, risk score trend Add overall asset risk score (weighted avg of top 5 finding scores)	Build scan history page in React: Table: Date / Target / Critical / High / Medium / Risk Score / Trend arrow Click row → go to that scan's findings Add Recharts line chart: risk score over time for each asset	Scan history table populates. Risk trend chart shows over multiple scans.
Day 39	Write GitHub Actions CI/CD workflow: <code>.github/workflows/ci.yml</code> On push: run pytest (backend tests) On push: run vitest (frontend tests) On merge to main: build + tag Docker images	Write frontend tests (vitest): FindingCard renders severity badge correctly Login form shows error on wrong password Findings table sorts by <code>priority_score</code> Fix any test failures.	CI runs green. Both backend and frontend tests pass automatically on push.
Day 40	Write docker-compose.prod.yml: Remove hot-reload volumes Add Caddy (reverse proxy + auto SSL) Add health checks to all services Test locally: <code>docker-compose -f docker-compose.prod.yml up</code>	Write production Dockerfile for frontend: Stage 1: <code>npm run build</code> → <code>dist/</code> Stage 2: <code>nginx:alpine</code> serves <code>dist/</code> Test: production image builds and serves UI correctly	<code>docker-compose.prod.yml</code> starts cleanly on laptop. Production frontend build serves correct UI.
Day 41	Buy domain (\$12, Namecheap) + Hetzner VPS (~€17/mo, CX31). Add DNS A record: <code>app.yourdomain.com</code> → server IP. SSH into server. Install Docker. Configure UFW firewall. git clone repo → deploy with <code>docker-compose.prod.yml</code>	Set up Caddy on server: <code>app.yourdomain.com</code> → React frontend (port 3000) <code>api.yourdomain.com</code> → FastAPI (port 8000) Caddy auto-issues Let's Encrypt SSL certificate Verify: <code>https://app.yourdomain.com</code> loads with green padlock	Platform live at <code>https://app.yourdomain.com</code> with valid SSL. API accessible at <code>https://api.yourdomain.com/health</code> .

Day	Sathish ■ — Backend support	Karthik ■ — Frontend + Deploy	■ Done When
Day 42	Set up monitoring: UptimeRobot: monitor app.yourdomain.com every 5 min Sentry: add SENTRY_DSN to .env → catches all Python errors DB backup cron: pg_dump daily at 02:00 → MinIO storage	Set up Sentry on frontend: npm install @sentry/react Add Sentry.init() → captures all JS errors in prod Test: trigger a test error, confirm it appears in Sentry dashboard	UptimeRobot monitoring active. Sentry captures test errors from both backend and frontend.
Day 43	Performance test on production server: Run 3 simultaneous scans. Watch: docker stats CPU should stay below 70%. RAM below 6GB. If needed: increase Celery concurrency from 4 to 8.	Write 2-page client onboarding guide (PDF): Page 1: How to log in + add a target + run a scan Page 2: How to read findings + create Jira ticket Include screenshots of every step.	3 concurrent scans complete. Server stable. Onboarding PDF ready to send to first client.
Day 44	Add multi-tenant org isolation test: Create 2 test orgs. Run scans on both. Confirm: Org A user cannot GET Org B findings (expect 403). Fix any isolation gaps found.	Polish UI for first client demo: Empty states: 'No scans yet — add your first target above' Loading skeletons while data fetches Mobile-responsive layout (Tailwind breakpoints)	Org isolation confirmed. UI has no empty state errors. Looks clean on mobile.
Day 45	Final security review of the platform itself: Run Nuclei against api.yourdomain.com Run Gitleaks on the repo (check for committed secrets) Fix any findings before first client onboarding.	Record a 5-minute demo video: Show: URL input → scan → AI findings → Jira ticket → PDF report Upload to Loom or YouTube (unlisted) Send link to potential first client with onboarding PDF	Platform scans itself cleanly. Demo video recorded and sent.
Day 46+	Monitor production daily: Check Sentry for new errors Check UptimeRobot for downtime Fix bugs reported by first client	Iterate on UI based on first client feedback. Improve any confusing UX. Add tooltips where needed. Start thinking about Phase 2 features.	First paying client using the platform. ■ Phase 1 complete — start planning Phase 2.

LAYER-BY-LAYER SUMMARY

Phase	Layer	Days	Key Milestone
Phase 1	Data Store	Days 1–3	All 5 DB tables created. Python can read/write findings.
Phase 2	Backend Skeleton	Days 4–10	Full API with auth, job queue, and mock scan working end-to-end.
Phase 3	Tool Layer	Days 11–22	All 6 scanners wired. Real findings in DB from real targets.
Phase 4	External APIs	Days 23–34	Every finding has CVSS, EPSS, KEV, ATT&CK;, AI summary, Jira ticket.
Phase 5	Frontend/Deploy	Days 35–46	Live on production. First client using it. First payment received.

Rule: never start a phase until the previous phase's completion signal is confirmed. Phase 3 adapters WILL FAIL if Phase 2 (the backend) is incomplete. Phase 4 AI enrichment WILL FAIL if Phase 3 findings do not exist in the DB. The dependency order is strict.

END OF DOCUMENT